

Linux SysAdmin & Kernel

Category: **Systems & OS Engineering** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Linux SysAdmin & Kernel**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

2. Core Concepts & Production Principles

- **Virtual File System (VFS) & Inode Mechanics:** The VFS layer abstracts underlying physical file systems. Inodes store file metadata (permissions, ownership, size, data block pointers) independently of filenames.

Production Application: Essential when diagnosing 'No space left on device' errors where disk space is reported available but 100% of inodes are exhausted due to millions of small session/cache files.

- **Process Lifecycle, States (R, S, D, Z) & Signal Handling:** Processes transition between Running (R), Interruptible Sleep (S), Uninterruptible Sleep (D - usually disk/NFS wait), and Zombie (Z). Controlled via signals like SIGTERM (15) and SIGKILL (9).

Production Application: Used in graceful blue-green deployment terminations where orchestrators wait for in-flight requests to complete before force-killing stuck processes.

- **Virtual Memory, Page Cache & OOM Killer:** Linux utilizes available RAM for page caching. When memory pressure exceeds physical RAM and swap, the OOM Killer invokes `oom_badness()` to terminate the highest-consuming process.

Production Application: SREs tune `/proc/sys/vm/swappiness` and set `oom_score_adj` on production databases to guarantee protection against unexpected OS terminations.

- **Systemd Architecture & cgroup Integration:** PID 1 init system providing parallel service activation, dependency tracking, socket activation, and strict cgroup resource constraints.

Production Application: Standardizing enterprise daemon deployment with automated restart policies, CPU quotas, and strict memory limits.

3. Exhaustive Command & Syntax Reference

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
find /var/log -type f -name '*.log' -mtime +14 -size +50M find -type f -name " -mtime + -size +	Locate rotated log files older than 14 days and larger than 50MB for archiving or cleanup. Options: [{ 'flag': '-type f', 'description': 'Filter for regular files only.' }, { 'flag': '-name "*.log"', 'description': 'Match filenames matching glob.' }, { 'flag': '-mtime +14', 'description': 'Modified more than 14 days ago.' }, { 'flag': '-size +50M', 'description': 'File size greater than 50MB.' }]	Run during off-peak maintenance windows or integrate with logrotate scripts. Pitfall: Omitting quotes around glob causing shell expansion; executing with -delete before verifying target files.
df -hT --exclude-type=tmpfs df [options] [path]	Inspect disk block allocation, mount points, available capacity, and filesystem types while excluding RAM tmpfs. Options: [{ 'flag': '-h', 'description': 'Human-readable format (GiB, MiB).'}, { 'flag': '-T', 'description': 'Display filesystem type.' }, { 'flag': '--exclude-type=tmpfs', 'description': 'Ignore temporary in-memory filesystems.' }]	Run immediately upon disk alert triage; check both block space (df -h) and inode space (df -i). Pitfall: Checking only block storage while ignoring inode depletion.
du -ah --max-depth=1 /var sort -hr head -n 10 du [options] sort -hr head -n	Identify the top 10 largest directory consumers on a disk partition. Options: [{ 'flag': '-a', 'description': 'Include files in addition to directories.' }, { 'flag': '-h', 'description': 'Human-readable size format.' }, { 'flag': '--max-depth=1', 'description': 'Limit reporting to immediate subdirectories.' }]	Execute from the root or /var partition to pinpoint the directory driving storage exhaustion. Pitfall: Omitting --max-depth, which causes du to output hundreds of thousands of lines on large filesystems.
journalctl -u nginx.service --since '1 hour ago' -p err..emerg --no-pager journalctl -u --since " -p [options]	Extract error-level and above systemd journal entries for a service from the past hour. Options: [{ 'flag': '-u', 'description': 'Filter by specific systemd unit.' }, { 'flag': '--since', 'description': 'Time window constraint.' }, { 'flag': '-p err..emerg', 'description': 'Priority filter (Error, Critical, Alert, Emergency).'}, { 'flag': '--no-pager', 'description': 'Direct output to stdout without interactive pager.' }]	Run in incident response runbooks to isolate exact crash logs without interactive prompts. Pitfall: Omitting --no-pager in automation scripts, causing CI or remote scripts to hang on interactive less prompt.

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
vmstat 1 5 vmstat [interval [count]]	Sample virtual memory, run-queue length, context switches, and block I/O at 1-second intervals. Options: [{flag: '1', 'description': 'Sample every 1 second.'}, {flag: '5', 'description': 'Terminate after 5 consecutive samples.'}]	Observe 'r' (run-queue) vs CPU cores, 'b' (blocked I/O), and 'si'/'so' (swap in/out). Pitfall: Relying on the first line of output which represents cumulative boot averages.

4. Security Hardening & Anti-Patterns

- Security Mandates:** Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.
- Critical Anti-Pattern:** Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.